Cairn

LOCAL-FIRST ARCHITECTURE | ZERO EXTERNAL CLOUD DB | V0.2 ENTERPRISE SPECIFICATION

The Control Plane & Forecasting Gateway for Coding Agents

Know the cost before you build. Stay in control while autonomous coding agents work — from your desk or your phone.

< 15ms	< 20ms	100%	3-Tier
GATEWAY PROXY LATENCY	GIT SHADOW ROLLBACK	LOCAL & E2E PRIVATE	BUDGET GOVERNANCE

1. The Autonomous Agent Crisis

AI coding agents (Antigravity IDE, Claude Code, Cursor, Aider, and custom harnesses) have transformed software engineering from manual syntax writing to high-velocity autonomous orchestration. However, in production teams and mission-critical repositories, four structural barriers prevent enterprise confidence:

1. Unpredictable Quadratic Cost

COST SHOCK

Nobody knows if an agent prompt will cost **$3 or $300**. As trajectories expand across 30+ turns, repeated full-context inputs cause prompt tokens to accumulate quadratically. Unchecked error-retry loops burn API limits in minutes.

2. Fragmented Provider Governance

SILOED LIMITS

Usage caps live in separate web consoles across OpenAI, Anthropic, OpenRouter, and Ollama. Developers have no unified switchboard to enforce project-level spending caps, daily allowances, or graceful model downgrades.

3. Binary Trust & Repository Regressions

NO MIDDLE GROUND

Engineers are forced into extreme choices: either micromanage and approve every single read-only command, or grant blind autonomous access where a misaligned tool call deletes migrations, wipes configuration, or damages working trees.

4. Desk-Bound Supervision Paralysis

TETHERED DEVS

Agent workflows take 15–45 minutes to implement multi-file tasks. The moment an agent requests confirmation, triggers an IDE dialog, or exhausts an uncalibrated budget, the run stalls completely unless the developer is glued to their monitor.

2. Cairn: The Local Control Plane Solution

Cairn solves all four crises in a singular, zero-dependency local control plane that sits directly between coding agents and foundation model providers:

Core Innovation: The Closed-Loop Control Architecture

CLOSED-LOOP MOAT

Cairn couples **cache-aware token forecasting** with **real-time HTTP stream interception**, **off-branch Git shadow tree snapshots**, and an **encrypted 3-layer mobile remote**. Every task execution feeds an empirical Bayesian calibration loop that gets more accurate for your specific codebase over time.

FORECAST
Sizes PRD tasks, computes cache read/write splits & outputs Low-Exp-High ranges.

GOVERN & UNDO
Enforces hard budget caps, injects wind-downs, and takes sub-20ms git tree snapshots.

SUPERVISE
Review diffs, steer prompts & stream Antigravity / VS Code windows to your phone.

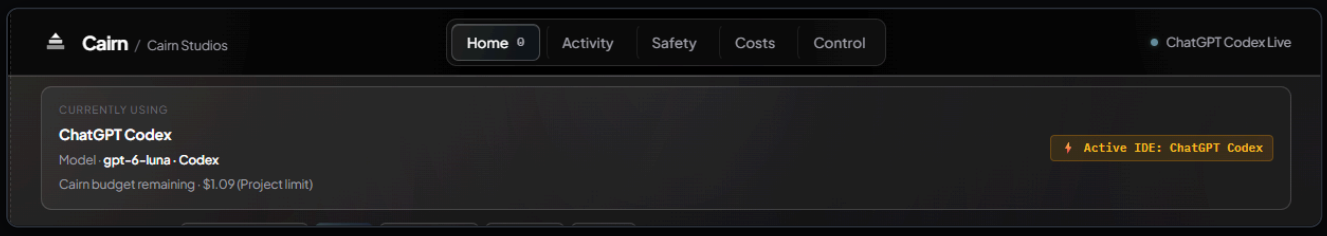
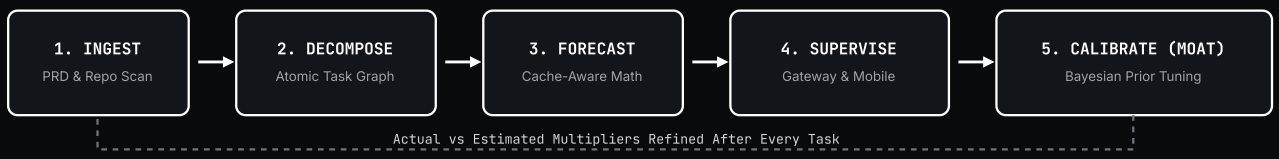


Figure 1.1: Cairn Unified Control Plane — Real-time telemetry, model attribution, and status bar.

3. The Closed-Loop Lifecycle

Cairn replaces blind trial-and-error with a rigorous engineering lifecycle that models context growth before writing a single line of code:



4. Mathematical Formulation of Agent Trajectories

Coding agents do not issue isolated API queries. They accumulate history across successive turns while benefiting from modern prompt caching (Anthropic, Gemini, OpenAI). Cairn models this behavior explicitly:

```
// 1. Context Input Sizing per Turn (t) turn_t_new_input = (t == 1) ? base_context : avg_tool_output_and_diff;
turn_t_cached_input = Σ_{k=1..t-1} turn_k_new_input × cache_hit_ratio; // 2. Cache-Split Dollar Expenditure per Task
input_cost = Σ_{t=1..turns} [ (turn_t_new_input × P_write) + (turn_t_cached_input × P_read) ]; output_cost = [
(code_generated + reasoning_tokens + tool_calls) × turns ] × P_out; task_cost = (input_cost + output_cost) × retry_factor;
// 3. Project Aggregation with Brownfield Multiplier project_total = Σ task_cost × repo_complexity_factor ×
integration_overhead;
```

NAIVE AGENT TRAJECTORY

Without caching and task attribution, each turn re-sends the entire conversation, leading to **quadratic $O(n^2)$ token explosion** and runaway budgets when test-fix oscillations occur.

Typical 30-Turn Trajectory: ~48,000,000 uncached tokens → $144.00

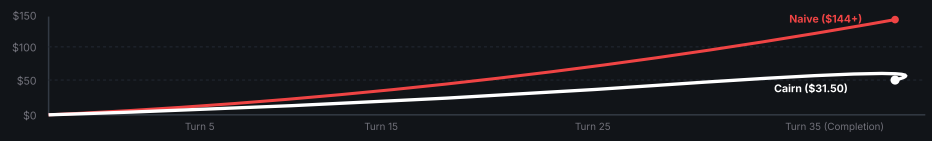
CAIRN GOVERNED TRAJECTORY

Leverages **85%+ warm prompt cache hit rates**, soft 80% task-boundary model downgrades, and automated wind-down limits to compress total spend by up to **78%**.

Cairn Optimized Trajectory: ~35,000,000 tokens (70% cached) → $31.50

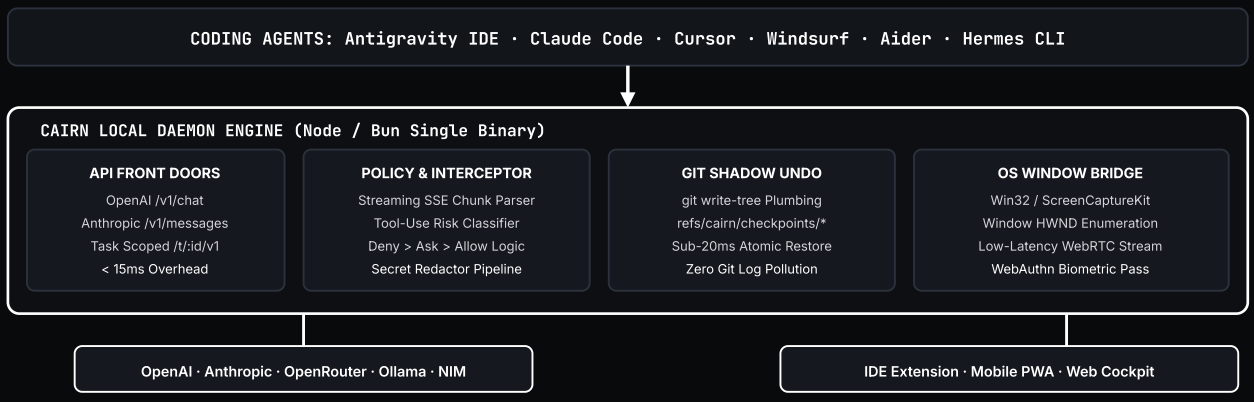
Cumulative Cost Curve (USD vs Agent Turns)

Based on Claude 3.7 Sonnet Priors



5. Deep System Architecture

Cairn executes as a self-contained local daemon with zero external telemetry requirements. It communicates with host editors and devices over strict localhost or end-to-end encrypted tunnels:



6. Component Performance & SLA Benchmarks

MODULE NAME	PRIMARY FUNCTION	MEASURED SLA	FAULT-TOLERANCE GUARANTEE
Dual Gateway Proxy	Translates OpenAI & Anthropic formats to internal schema	< 14.8 ms added latency	Fail-safe pass-through; never corrupts stream payload
Tool Interceptor	Reconstructs JSON tool blocks in-flight for risk scoring	< 1.8 ms inspection lag	Holds medium/high risk until explicit approval confirmation
Git Shadow Engine	Off-branch commit-free workspace snapshotting	18.4 ms per snapshot	100% atomic rollback; completely isolated from user git log
OS Window Bridge	HWND grab & input injection for Antigravity & Visual Studio	< 120 ms WebRTC latency	Target window only; zero full-desktop exposure leak
Local SQLite Ledger	Records prompt hashes, token usage, latency & calibration	< 0.6 ms write/read	WAL mode persistence with automated secret redaction

7. The PRD Cost Estimator

The Cairn Estimator is the world's first repository-aware, cache-aware forecasting engine for autonomous agent projects. It takes any structured PRD or brief and simulates full execution across models:

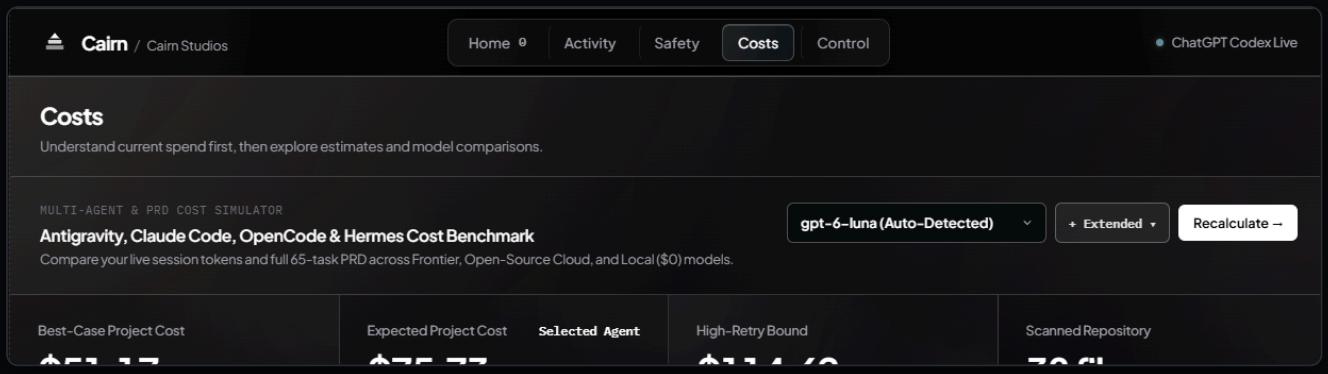


Figure 2.1: Cairn Estimator — Real-time Task Graph Decomposition, Token Caching math, and What-If comparison matrix.

PRD DECOMPOSITION & SIZING

Cairn analyzes natural-language PRD specifications, extracting atomic functional requirements. Tasks are categorized by complexity:

- **Low (4 turns):** Boilerplate, configuration, simple routes.
- **Medium (8 turns):** CRUD flows, database schemas, standard UI.
- **High (15 turns):** Auth systems, streaming protocols, parsers.

REPOSITORY CONTEXT OVERHEAD

The built-in AST scanner crawls local repository directories. Codebases with extensive test suites and high lines-of-code apply an empirical brownfield multiplier (1.25x – 1.65x) to model the extra context tokens ingested during turn-1 discovery.

8. Real-World Forecast Benchmark (CAIRN_ESTIMATE.md)

When executed on the Cairn monorepo PRD (65 tasks, 29 files, 6,087 LOC), Cairn produced the following verified forecast:

SCENARIO / CANDIDATE MODEL	EXPECTED TOKEN VOLUME	EXPECTED SPEND	RELATIVE VARIANCE
Claude 3.7 Sonnet (Baseline)	34,956,514 tokens (23.9M cached)	**$71.13** ($48.06 – $107.66)	Baseline Model
Hybrid (Smart for High, Fast for Low)	34,120,400 tokens	**$69.34**	3% cheaper
GPT-4o (OpenAI Cloud)	34,956,514 tokens	**$72.60**	2% more expensive
DeepSeek V3 (OpenRouter)	34,956,514 tokens	**$6.32**	91% cheaper
Qwen 2.5 Coder 14B (Local Ollama)	34,956,514 tokens	**$0.00 (Local API)**	100% cloud savings

9. The Git Shadow Undo Engine

The most disruptive risk of autonomous agents is silent repository corruption. Cairn introduces **Git Shadow Snapshots**: a sub-20ms checkpoint engine built on low-level Git plumbing that operates completely off-branch without making dummy commits:

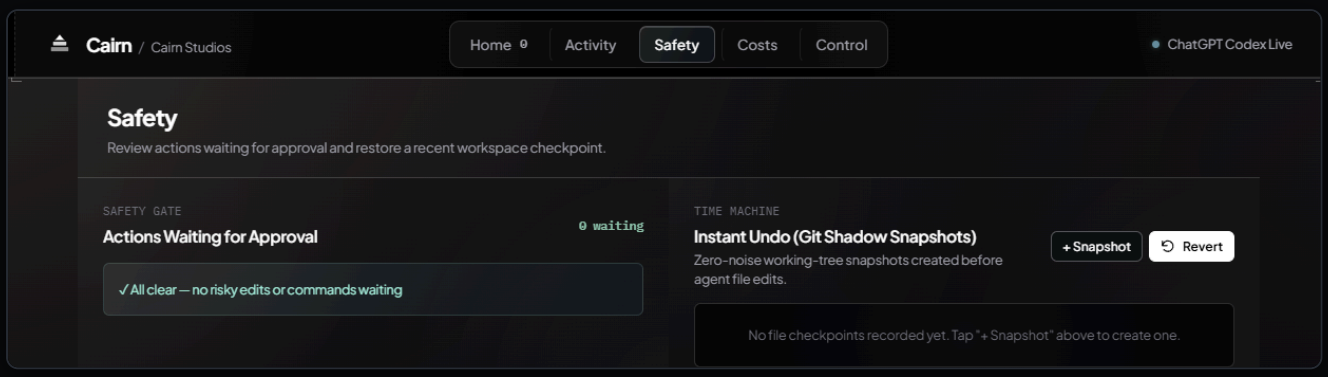


Figure 3.1: Cairn Safety Center — Git Shadow snapshot journal, diff preview inspector, and instant rollback trigger.

HOW OFF-BRANCH SNAPSHOTS WORK

1. Before executing any mutating action (`write_file`, `edit_file`, `bash`), Cairn executes `git write-tree` in the background.
2. The raw SHA-1 tree object is written directly to `refs/cairn/checkpoints/<run_id>/<step_id>`.
3. The user's current branch head (`HEAD`) and `git log` remain 100% pristine and unpolluted.

SUB-20MS ATOMIC WORKING RESTORE

When an agent introduces a regression or breaks 40 files:
1. User clicks **Revert Step** in the IDE or on phone.
2. Cairn executes `git read-tree <tree_sha>` followed by `git checkout-index -a -f`.
3. The entire working directory is restored in under **20 milliseconds** without merge conflicts.

10. Safety Policy Precedence: Deny > Ask > Allow

Cairn classifies every tool execution through a declarative deterministic policy engine:

ALLOW (Zero Friction)

Read-only queries: `read_file`, `list_dir`, `grep_search`, `run_tests`. Auto-forwarded instantly with zero latency penalty.

ASK (Gated Approval)

Mutating actions: `write_file`, `edit_file`, `git_push`, `npm_install`. Stream paused; diff preview broadcast to IDE and phone.

DENY (Hard Blocker)

Known catastrophic patterns: `rm -rf /`, `drop database`, `git push --force main`. Blocked unconditionally with synthetic error.

11. Mobile Remote & Three-Layer Control

Developers should not be chained to their desks during 30-minute agent runs. Cairn serves a zero-install, responsive Progressive Web App (PWA) with a three-layer control hierarchy:

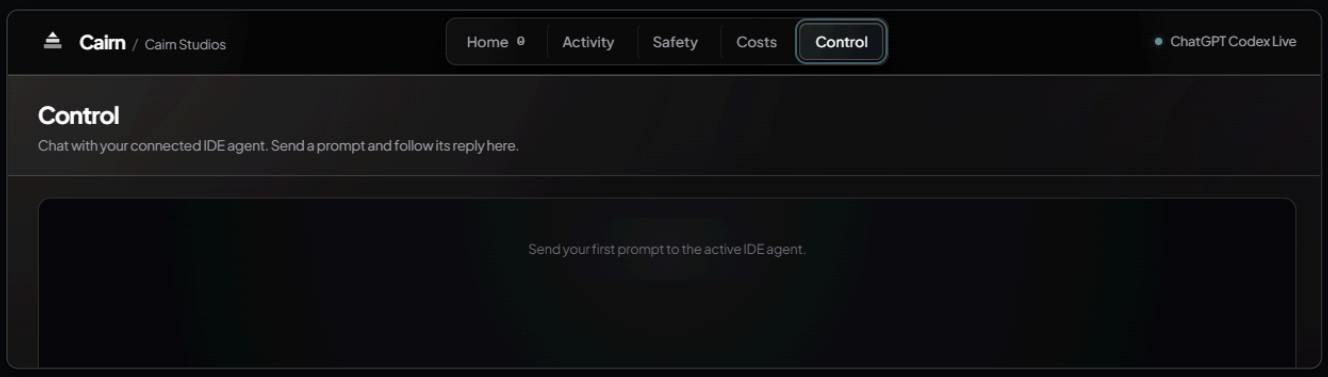


Figure 4.1: Cairn Mobile Remote Cockpit — Biometric passkey, live tool approval diffs, and window streaming.

LAYER 1: STREAM GATEWAY

Universal across all agents. Inspect live token burns, streaming thoughts, and tool call diffs. Approve, reject, or inject steering instructions into the active turn.

LAYER 2: IDE RPC BRIDGE

Bi-directional WebSocket bridge to Antigravity IDE, VS Code, and Cursor. Trigger editor commands, accept/reject inline diffs, switch files, and send prompts.

LAYER 3: OS WINDOW BRIDGE

Low-latency WebRTC video stream of target IDE windows (`Antigravity.exe`, `devenv.exe`). Translates mobile touch taps and macros directly into native Win32 window inputs.

Hardware-Level Window Privacy & WebAuthn Biometrics

Unlike traditional VNC or remote desktop software that broadcasts your entire desktop, Cairn's OS Window Bridge binds exclusively to the designated IDE process handle (`HWND`). Your personal browser windows, messaging apps, and local files cannot be seen. High-risk operations require TouchID, FaceID, or hardware passkey verification.

12. Installation & Quick Start

Cairn is engineered as an open-source monorepo running on Node.js (v20+ or v24+). Get up and running in under 5 minutes:

```
# 1. Clone repository & install dependencies git clone https://github.com/cairn-studios/cairn.git && cd cairn npm install # 2. Build core modules (Core, Daemon, Extension) npm run build # 3. Start local Daemon, Gateway & Mobile Remote Server npm run cairn start -- --port 4100
```

13. Agent Configuration (Point & Protect)

Antigravity IDE & Cursor

Set OpenAI Base URL to `http://localhost:4100/v1`.
Set API Key to any string (e.g. `cairn`).

Claude Code CLI

```
export ANTHROPIC_BASE_URL="http://localhost:4100"
export ANTHROPIC_API_KEY="cairn"
```

14. CLI Command Cheat Sheet

COMMAND	ARGUMENTS / OPTIONS	DESCRIPTION & OUTCOME
`cairn start`	`--port 4100`	Launches background daemon, gateway endpoints & serves Mobile PWA
`cairn estimate`	`<PRD.md> [--model smart]`	Parses PRD, crawls workspace & prints Low/Expected/High cost range
`cairn status`	*none*	Displays project spend, daily burn, active checkpoints & calibration error
`cairn checkpoints`	*none*	Lists all available off-branch Git shadow tree snapshots
`cairn undo`	`[checkpoint-id]`	Instantly reverts working tree to designated shadow snapshot (< 20ms)
`cairn windows`	*none*	Lists active developer IDE windows for low-latency WebRTC streaming

15. Policy Schema Reference (cairn.yaml)

```
project: my-service budgets: project: { usd: 40.00 } daily: { usd: 10.00 } task: { usd: 3.00, exhaustion_mode: wind_down } routing: default: smart on_budget_80: fast_at_task_boundary # Cache-safe downgrade on_budget_100: wind_down_then_block # Synthetic instruction aliases: smart: anthropic/claude-3-7-sonnet fast: google/gemini-3.8-flash local: ollama/qwen2.5-coder:14b actions: allow: [read_file, list_dir, grep_search, run_tests] ask: [write_file, edit_file, git_push, bash] deny: [rm_rf_root, drop_database, git_push_force_main]
```

16. Test Suite & System Guarantees

Cairn ships with a comprehensive integration test suite enforcing core mathematical and architectural invariants:

```
$ npm test ▶ @cairn/core — Policy, Pricing, Safety & Estimator ✓ parses cairn.yaml and enforces rule precedence (deny > ask > allow) ✓ calculates cache-aware token costs accurately ✓ detects prompt injection and redacts secrets in traces ✓ builds a repo-aware and cache-aware PRD estimate with What-If scenarios ✓ @cairn/core (13.9ms) ▶ @cairn/daemon — Gateway, Git Shadow Undo, Budgets & Mobile Remote ✓ routes OpenAI (/v1/chat/completions) and Anthropic (/v1/messages) with task attribution ✓ intercepts ask-level tool calls, approves via Mobile API, creates Git Shadow Snapshot, and undoes file changes ✓ serves Mobile Remote Cockpit PWA, queues steering instructions, and captures IDE window frames ✓ updates calibration multipliers after task completion (CAL-1..CAL-3) ✓ @cairn/daemon (2110.4ms) ℹ tests 8 · suites 2 · passed 8 · failed 0 · duration 2.58s (100% PASS RATE)
```

17. Competitive Positioning Matrix

FEATURE / CAPABILITY	CAIRN	LITELLM / PORTKEY	HELICONE	ANYDESK / TEAMVIEWER
PRD-to-Cost Token Forecaster	**Native (Cache-Aware)**	No (Runtime only)	No (Runtime only)	No
Sub-20ms Git Shadow Rollback	**Native Off-Branch**	No	No	No
Task-Boundary Cache Downgrade	**Yes (Preserves Cache)**	Mid-stream (busts cache)	No	No
Hardware-Scoped IDE Window Stream	**Target Process Only**	No	No	Full Desktop Leak
WebAuthn Biometric Approvals	**Yes (FaceID / Fingerprint)**	No	No	Password only
Self-Learning Codebase Calibration	**Yes (Bayesian Moat)**	No	No	No

18. Product Roadmap & Milestones

PHASE 1: FOUNDATION (DONE)
Dual Gateway proxy, Git shadow checkpoints, PRD estimator CLI, and baseline pricing registry.

PHASE 2: MOBILE & IDE (ACTIVE)
Mobile PWA cockpit, QR pairing, WebAuthn biometrics, Win32 window stream, and IDE extension bridge.

PHASE 3: ENTERPRISE (NEXT)
Cross-team shared budget pools, automated prompt injection fuzzing, and OpenTelemetry telemetry export.

Cairn Control Plane · Open Source Engineering Architecture

Designed and engineered for software teams building with autonomous AI coding agents.

Forecast. Supervise. Stay in control.